Brute force :-

```
public static int findDays (int weights[], int cap) {
    int days = 1;
    int load = 0;
    int n = weights.length;

    for (int i = 0; i<n; i++) {
        if (load + weights[i] > cap) {
            days += 1;
            load = weights[i];
        }
        else {
            load += weights[i];
        }
    }
    return days;
}
```

TC - O(N*(sum(weights[]) - max(weights[])+1))
SC - O(1)

```
public static int leastWeightCapacity (int weights[], int d) {
    int max = Integer.MIN_VALUE;
    int sum = 0;

    for (int i = 0; i<weights.length; i++) {
        sum += weights[i];
        max = Math.max (max, weights[i]);
    }

    for (int i = max; i <= sum; i++) {
        if (findDays (weights, i) <= d) {
            return i;
        }
    }
    return -1;
```

05. ~~Sudoku solver~~ -

Optimal (Binary Search) Dry Run :-

Dry-run Raw Page Here

Ex ⇒ [3, 2, 2, 4, 1, 4] (0 1 2 3 4 5), days = 3

5 + 6 + 5

Iteration-01 :- low = max high = sum

low = 4 high = 16

mid = 16+4/2 ⇒ 10

reqDays () load = 0, days = 1

i = 0 ⇒ load = load + weights(i)

= 0 + 3 ⇒ 3 < mid

i = 1 ⇒ load = 3+2 ⇒ 5 < mid

i = 2 ⇒ load = 5+2 ⇒ 7 < mid

i = 3 ⇒ load = 7+4 ⇒ 11 > mid

days = 2 load = 4

i = 4 ⇒ load = 4+1 ⇒ 5 < mid

i = 5 ⇒ load = 5+4 ⇒ 9 < mid

days == 2 < days high = mid - 1

Iteration 02 :- low = 4 high = 9

mid = 9+4/2 ⇒ 13/2 ⇒ 6

reqDays () load = 0, days = 1

i = 0 ⇒ load = 0+3 ⇒ 3 < mid

i = 1 ⇒ load = 3+2 ⇒ 5 < mid

PAGE NO
DATE / /

i=2 = load = 5+2 = 7 ~~>~~ mid days = 2 (load>)2
i=3 = load = ~~5~~ +4 = ~~6~~ 7 = mid ~~days = 2~~ ~~load = 4~~

i=4 = ~~6~~ +1 ⇒ ~~7~~ ⇒ mid days=3 load=1
i=5 = ~~5~~ +4 ⇒ ~~5~~ < mid ~~days=2~~ ~~load=4~~

days (3) == 2, high = mid − 1;

Iteration 03

low = 4, high = 6
mid = 6+4 ⇒ 10/2 ⇒ 5
reqDays() load = 0, days = 1

i=0 = load = 3 < 5
i=1 = 3+2 ⇒ 5 == 5
i=2 = 5+2 ⇒ 7 > 5 days ⇒ 2 load ⇒ 2
i=3 = 2+4 ⇒ 6 > 5 days ⇒ 3 load ⇒ 4
i=4 = 4+1 ⇒ 5 == 5
i=5 = 5+4 ⇒ 9 > 5 days ⇒ 3 load ⇒ 4

(3) days == 3 high = mid − 1

Iteration 04 low = 4 Ans. high = 4
mid = 4+4 ⇒ 8/2 ⇒ 4
reqDays() load = 0, days = 1
i=0 ⇒ load ⇒ 3 < 4
i=1 ⇒ 4+2 ⇒ 6 > 4 days = 2 load ⇒ 2
i=2 ⇒ 2+2 ⇒ 4 == 4
i=3 ⇒ 4+4 ⇒ 8 > 4 days = 3 load ⇒ 4
i=4 ⇒ 4+1 ⇒ 5 > 5 days ⇒ 4

- Optimal Code (Binary Search) -

```
public static int findDays(int weights[], int cap) {
    int days = 1;
    int load = 0;
    int n = weights.length;

    for(int i = 0; i<n; i++) {
        if(load + weights[i] > cap) {
            days += 1;      // move to next day
            load = weights[i]; // Load the weight
        }
        else {
            load += weights[i];
        }
    }
    return days;
}


public static int leastWeightCapacity(int weights[],
    int d) {

    int low = Integer.MIN_VALUE;
    int high = 0;

    for(int i = 0; i<weights.length; i++) {
        high += weights[i];
        low = Math.max(low, weights[i]);
    }
```

```
while (low <= high) {
    int mid = (low) + (high-low)/2;

    int numberOfDays = findDays(weights, mid);

    if( numberOfDays <= d) {
        high = mid-1; //eliminate right half.
    }
    else {
        low = mid+1; //Eliminate left half
    }
}
return low;
}
```

TC- O(N* log(sum(weights[])- max(weights[]) +1))

SC- O(1)

20. Kth missing Positive Number –

arr[] = [2, 3, 4, 7, 11] k = 5

ans ⇒ 9

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

1 2 3 4 5

Ex. :– [2, 3, 4, 7, 11] k = 5

Iteration 01 :– (i) i = 0 == 2 <= k(5)

k = 6

(ii) i = 1 == 3 <= k (6)

k = 7

(iii) i = 2 == 4 <= k (7)

k = 8

(iv) i = 3 == 7 <= k (8)

k = 9

(v) i = 4 == 11 > k (9)

return k == 9

Ans.

Code:- (Brute Force):-

~~N-Queens~~

```
public Static int missingK(int vec[], int n, int k){
    for (int i = 0; i<n; i++){
        if (vec[i] <= k) {
            k++;
        }
        else {
            break;
        }
    }
    return k;
}
```

TC - O(N)
SC - O(1)

OPTIMAL EXPLAINATION -

idx - 0 1 2 3 4

Idealy - [2, 3, 4, 7, 11]

1 2 3 4 5

missing = 1 1 1 3 6

idx

missing = arr[mid] - (mid+1)

Dry-Run Code (Optimal):-

Ex. = [2, 3, 4, 7, 11] k = 5

missing → 1 1 1 3 6

How did we find the missing numbers?

ideally

at 2 there should be 1 ⇒ 2-1 ⇒ 1
at 3 there should be 2 ⇒ 3-2 ⇒ 1
at 4 there should be 3 ⇒ 4-3 ⇒ 1
at 7 there should be 4 ⇒ 7-4 ⇒ 3
at 11 there should be 5 ⇒ 11-5 ⇒ 6

Iteration 01 :- low = 0th index high = last index

mid = $\frac{0+4}{2}$ ⇒ 2

[2, 3, 4, 7, 11]
0 1 2 3 4
low mid high

missing (1) < k (5)

low = mid + 1

Iteration 02 :- low = 3 high = 4

mid = $\frac{3+4}{2}$ ⇒ $\frac{7}{2}$ ⇒ 3

[2, 3, 4, 7, 11]
0 1 2 3 4
low high
mid

missing (3) < k (5)

low = mid + 1

Iteration 03 ⇒ low = 4 high = 4

mid = $\frac{4+4}{2}$ ⇒ 8/2 ⇒ 4

[2, 3, 4, 7, 11]
0 1 2 3 4

high
low
mid

missing (6) > k (5) high = mid − 1

Iteration 04 low > high (crossed)

{opposite polarity}

k ⇒ 5 → 3 → 2

arr(high) = 7 missing ⇒ 3 more = 2

arr(high) ⇒ 8 + 1 + 1 } more (2)

9

Ans.

// Can Not Apply typical BS
// Can Not Apply BS on Ans.

Optimal (Binary Search) Code:-

```
public static int missingK(int vec[], int n, int k){
    int low = 0;
    int high = n-1;

    while (low <= high) {
        int mid = low + (high-low)/2;

        int missing = vec[mid] - (mid+1);

        if (missing < k) {
            low = mid + 1;
        }

        else {
            high = mid-1;
        }
    }
    return k + high + 1;
}
```

arr[high] + more(k - missing)
⇒ arr[high] + arr[high] − (high + 1)
(−high − 1)
⇒ arr[high] + k − arr[high] + high + 1
⇒ high + 1 + k ⇒ low + k

{ figure out two nearby indexes }

TC- O(log N)
SC- O(1)

Ex. [4, 7, 9] (arr[high] + more)
high (−1) idx ⇒ that why will not work
k = 3
k + high + 1; (return)

~~ideally [2, 3, 4, 7, 11]~~
~~0 1 2 3 4~~
~~missing~~
~~7 − 3 ⇒ 4~~ ~~5 4~~ Now

Pattern changed at the Qs. ⇒ (min) max ⇒ max (min)

PAGE NO.
DATE: / /

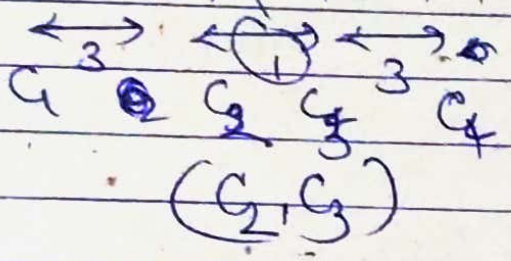
21. Aggressive Cows –

arr[] = [0, 3, 4, 7, 10, 9] cows = 4 ans = 3

~~Bin~~ place these cows in such a way so that the minimum distance between any of the 2 cows is ~~minimum~~ min (maximum)..

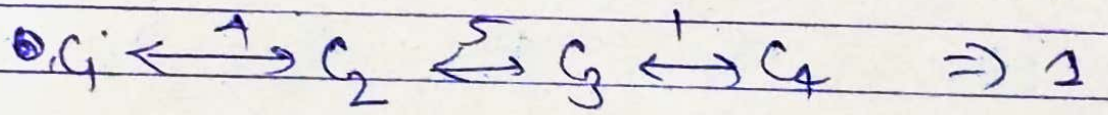
{minimum distance will always between two consecutive cows}

0, 3, (4, 7, 9, 10



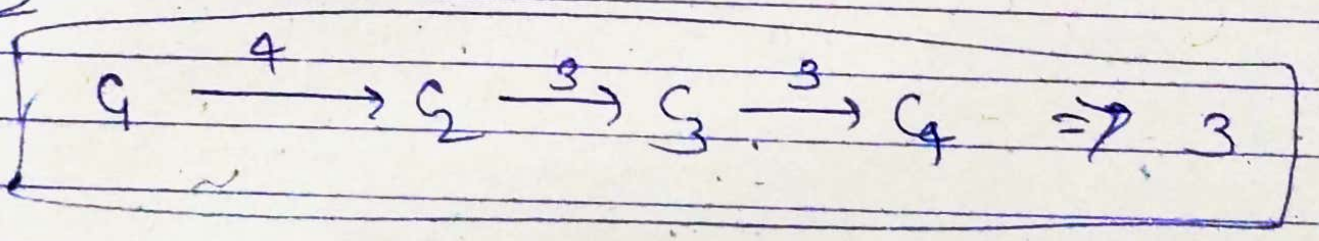
⇒ 1

(C2, C3)



⇒ 1

Ans.



⇒ 3

~~of~~ ~~N~~ ~~Queens~~ ~~II~~

Brute Force :- Dry - Run

Sort the Array first

⇒ arr[] = [0, 3, 4, 7, 9, 10] cows = 4

① dist = 1 — cow1 cow2 cow3 cow4 ⇒ 1 ✓

② [illegible]

② dist = 2 — [0, 3, 4, 7, 9, 10]

cow1 cow2 cow3 cow4

dist ⇒ 3, 4, 2 — 2 ✓

③ dist = 3

[0, 3, 4, 7, 9, 10]

cow1 cow2 cow3 cow4

3, 4, 3

3 ✓

④ dist = 4

[0, 3, 4, 7, 9, 10]

cow1 cow2 cow3 cow4

4 ✗ (not possible)

arr() = [0 , 3 , 4 , 7 , 9 , 10] cows = 4

(c1) min ⟵ ⟶ max (c2)

★ [max-distance where I can place two cows is (max-min)]

[range → max-min → (1, max-min)] arr[n-1] - arr[0]

Iteration 01 :- i = 1 canWePlace(nums, m, i)

arr[0]

(i) j = 1 last = 0 , cntCows = 1

(3-0) → 3 > 1

cntCows = 2

last = arr[1]

(ii) j = 2

(4-3) == 1 continue

cntCows = 3

last = arr[2]

(iii) j = 3 (7-4)

(4-3) == 3 > 1

cntCows = 4

last = arr[3] == 7

(iv) j = 4

(9-7) == 2 > 1

cntCows = 5

last = 9

(v) j = 5

(10-9) == 1 == 1

continue cntCows = 6 last = 10

cntCows (6) >= cows

Iteration 02 :- i = 2

Brute force Code:-

```
Public int boolean canWePlace(int stalls[], int
                    dist, int cows){
    int  n = stalls.length;
    int cntCows = 1;
    int last = stalls[0];
          ↳ first cow place here
    for(int i = 1; i<n; i++){
        if (stalls[i] - last >= dist){
            cntCows++;
            last = stalls[i];
        }
        if (cntCows >= cows) return true;
    }
    return false;
}
```

TC - O(NlogN) + O(N*(max(stalls[])-min(stalls[])))
SC - O(1)

```
public static int aggressiveCows(int stalls[], int k){
    int n = stalls.length;

    Arrays.sort(stalls);

    int limit = stalls[n-1] - stalls[0];

    for(int i = 1; i<= limit; i++){
        if (canWePlace(stalls, i, k) == false){
            return (i-1);
        }
    }
    return limit;
```

Dry Run (Optimal Approach) -

[0, 3, 4, 7, 10, 9] ⇒ Sort cows = 4

[0, 3, 4, 7, 9, 10]

⇒ Iteration 01 :- low = 0 high = 10 (max)

mid = 10/2 = 5

canPlace(nums, 5, cows = 4

j = 1 → cntCow = 1, last = 0

- arr[j] - last ⇒ 3 - 0 ⇒ 3 >= 1
- cntCow = 2, last = 3

j = 2 → cntCow = 2 last = 3

- arr[j] - last ⇒ 4 - 3 ⇒ 1 >= 1
- cntCows = 3, last = 4

j = 3 → cntCow = 3, last = 4

- arr[j] - last ⇒ 7 - 4 ⇒ 3 >= 1
- cntCow = 4, last = 7

:

j = 4 → cntCow = 4, last = 7

- arr[j] - last ⇒ 9 - 7 ⇒ 2 >= 1
- cntCow = 5, last = 9

j = 5 cntCow = 5, last = 9

- arr[j] - last ⇒ 10 - 9 ⇒ 1 >= 1
- cntCow = 6, last = 10

Drop...

cntCow >= cows { low = mid + 1

[0, 3, 4, 7, 9, 10]
0 1 2 3 4 5

low = 6 high = 10 mid = 16/2 = 8

Iteration 02 :- canPlace(nums, 8, cows)

- j=1 cntCow = 1, last = a[0]
  - a[j] - last ⇒ 3 - 0 ⇒ 3 < 8 (j++) last = a[j]
- j=2
  - a[j] - last ⇒ 4 - 3 ⇒ 1 < 8 (j++) (last = a[j])
- j=3
  - a[j] - last ⇒ 7 - 4 ⇒ 3 < 8 (j++) (last = a[j])
- j=4
  - a[j] - last ⇒ 9 - 7 ⇒ 3 < 8 (j++) (last = a[j])
- j=5 (j reached end) Stop...
  - a[j] - last ⇒ 10 - 9 ⇒ 1 < 8 (j++) (last = a[j])

cntCows (1) < cows high = mid - 1.

Iteration 03 :- low = 6, high = 7 mid = 13/2 = 6

canPlace(nums, 6, cows)

- j=1 cntCow = 1, last = 0
  - 3 - 0 ⇒ 3 < 6

Code:- Optimal (Binary Search)

```
public static boolean canWePlace(int stalls[], int dist, int cows){
    int n = stalls.length;
    int cntCows = 1;
    int last = stalls[0];

    for(int i = 1; i<n; i++){
        if( stalls[i] - last >= dist){
            cntCows++;
            last = stalls[i];
        }

        if (cntCows >= cows){
            return true;
        }
    }
    return false;
}


public static int aggressiveCows(int stalls[], int k){
    int n = stalls.length;
    Arrays.sort(stalls);

    int low = 1;
    int high = stalls[n-1] - stalls[0];
```

```
while (low <= high) {
    int mid = low + (high - low) / 2;

    if (canWePlace(stalls, mid, k) == true) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

return high;
}
```

TC- O(NlogN) + O(N * log(max(stalls[]) - min(stalls[])))

SC- O(1).

{ low is in possible polarity.
high is pointing to not-possible
Binary Search do it opposite polarity }
so high will be in possible place
while low will be in opposite place }

⇒ Min (max)

22. Allocate Books - Book should allocate to the given student...

arr[] = [25, 46, 28, 49, 24] Student = 4

① 25 | 46 | 28 | 49, 24 → 73

② 25 | 46 | 28 49 | 24 → 77

③ ~~25 | 46 | 28~~
25 | 46, 28 | 49 | 24 → 74

④ ans 25, 46 | 28 | 49 | 24 → 71 (min)

return

Brute force:-

{ high → sum } Range

① If the no. of students is greater than the no. of books return...

② Range :- [Biggest No. → Sum of given Arr]

at min one student can take → Biggest No.

at max one student can take → Sum of given Arr.

Brute Force Dry Run:—

⇒ [25, 46, 28, 49, 24] student = 4

↓ Sort

~~[25, 28,~~ ~~[24, 25, 28, 46, 49]~~

min = (49) max = sum of given arr.

(1) (49) [25, 46, 28, 49, 24]

(1) 24 + 46 > 49 ⇒ (2)

~~24 + 25 + 28 > 49~~ (2)

(2) pages = 46

[25, 46, 28, 49, 24]

28 + 46 > 49 (3)

(3) pages = 28

[25, 46, 28, 49, 24]

~~46 + 49 > 49~~ (4)

28 + 49 > 49

(4) pages = 49

[25, 46, 28, 49, 24]

49 == 49 + 24 > 49 (5)

Students == 4 ✓

(5) pages = 24

Students ⇒ 5

Brute force code :-

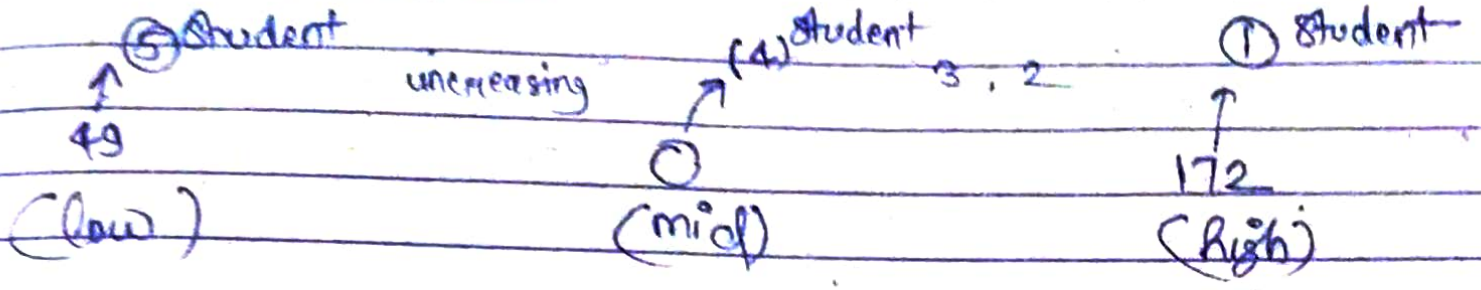
```
public static int countStudents (int arr[], int pages){
    int n = arr.length;
    int Students = 1;
    long pagesStudent = 0;

    for (int i=0; i<n; i++) {
        if (pagesStudent + arr[i] <= pages) {
            pagesStudent += arr[i];
        }
        else {
            Students++;
            pagesStudent = arr[i];
        }
    }
    return Students;
}

public static int findPages (int arr[], int n, int m) {
    // book allocation impossible
    if (m > n) {
        return -1;
    }

    int low = Math.max(arr);
    int low = Arrays.stream(arr).max().getAsInt();
    int high = Arrays.stream(arr).sum();
```

```
for(int pages = low; pages <= high; pages++){
    if(countStudents(arr, pages) == m){
        return pages;
    }
}
return low;
}
```

TC - O(N*(sum(arr[]) - max(arr[]) + 1)),

SC - O(1)

Optimal Code Dry Run:-



arr() = [25, 46, 28, 49, 24]

Optimal (BS) Code:-

```
public static int countStudents (int arr[], int pages) {
    int students = 1;
    long pagesStudent = 0;

    for (int page : arr) {
        if (pagesStudent + page <= pages) {
            // add pages to current student
            pagesStudent += page;
        }
        else {
            // add pages to next student
            students++;
            pagesStudent = page;
        }
    }
    return students;
}

public static int findPages(int arr[], int n, int m) {
    // book allocation impossible
    if (m > n) return -1;

    int low = Arrays.stream(arr).max().getAsInt();
    int high = Arrays.stream(arr).sum();

    while (low <= high) {
        int mid = low + (high-low)/2;
```

```
int students = countStudents(arr, mid);
if (students > m) {
    low = mid + 1;
}
else {
    high = mid - 1;
}
}
return low;
}
```

TC - O(N * log(sum(arr[])) - max(arr[]) + 1))

SC - O(1)

PAGE NO
DATE:

23. Split Array - Largest Sum :- / ~~Partk~~ Painters Partition

arr[] = [10, 20, 30, 40] k = 2, ~~ans~~

Explanation :- ① [10] [20, 30, 40) → 90

② [10, 20) [30, 40) → 70

③ [10, 20, 30) [40] → 60

min of the Maximum

any.

[This problem is same as the Book Allocation Problem. Everything will be the same, logic, Concept & Code will be same.]

24. Minimize Max Distance to Gas Station -

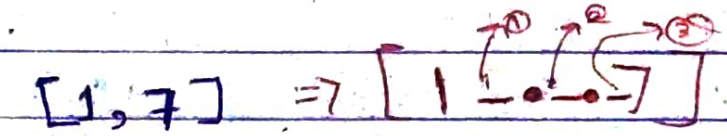
Note :- Answer within 10^{-6} of the actual answer will be accepted.

Input :-

arr[] = [1, 2, 3, 4, 5] k = 4 (Stations).

output :-

ans = 0.5 (dist).

[1, 7] ⇒ 

Explanation :-

arr[] = [1, 2, 3, 4, 5] k = 4

Ex. ⇒

arr[] = [1, 13, 17, 23] k = 5

1 —3— 4 —3— 7 —3— 10 —3— 13 —2— 15 —3— 17 —3— 20 —3— 23

$\frac{12}{4} = 3$ $\frac{17-13}{2} = 2$ $\frac{23-17}{2} = 3$

How many placed + 1

max = 3 Ans.

{ place in between given Station }

Code

Brute force:- (Hard Problem)...

PAGE NO.
DATE: / /

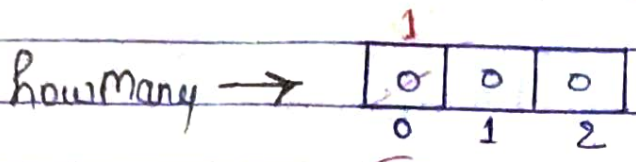
```
public static double minimiseMaxDistance (int arr[], int k){
    int n = arr.length;
    int howMany[] = new int[n-1];

    // Pick and Place k gas Station
    for(int gasStations = 1; gasStations <= k; gasStations++){
        double maxSection = -1;
        int maxInd = -1;
        for(int i = 0; i<n-1; i++){
            double diff = arr[i+1] - arr[i];
            double sectionLength = diff / (double) (howMany[i]+1);

            if (sectionLength > maxSection){
                maxSection = sectionLength;
                maxInd = i;
            }
        }
        howMany[maxInd]++;
    }
    double maxAns = -1;
    for(int i=0; i<n-1; i++){
        double diff = arr[i+1] - arr[i];
        double sectionLength = diff / (double) (howMany[i] + 1);
        maxAns = Math.max (maxAns, sectionLength);
    }
    return maxAns;
}
```

TC- O(k * N) → k: no. of gas Station placed; N: size of the given arr

SC- O(n-1) → using arr to keep track of placed stations.

Brute force Dry-Run :-

arr[] = [1, 13, 17, 23] K = 5

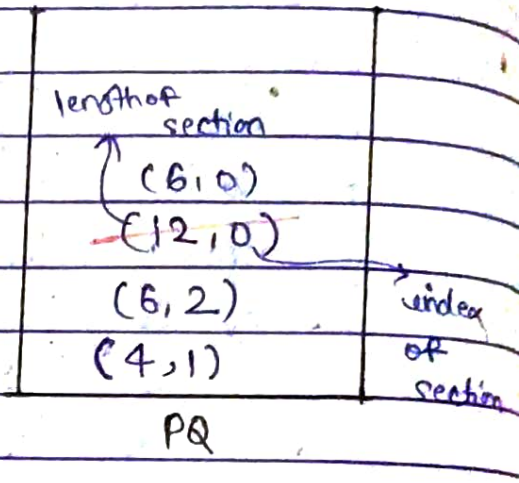


12 4 6

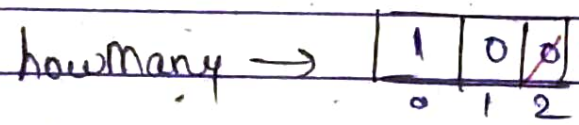
max

(1) (12, 0)

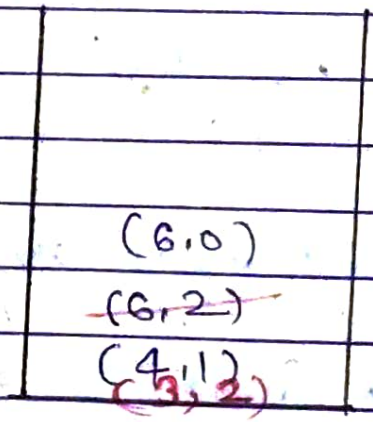
$\frac{12}{1+1} \Rightarrow \frac{12}{2} \Rightarrow 6$



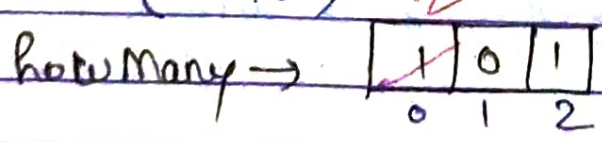
(2) (6, 2)



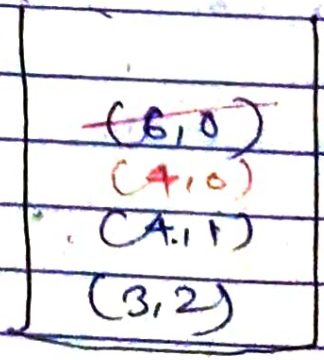
$\frac{6}{2} \Rightarrow 3$



(3) (6, 0)



$\frac{12}{3} \Rightarrow 4$



PAGE NO.
DATE: / /

Better Dry Run :-

(4) (4,1)

howMany → | 2 | 0 | 1 |
0 1 2

$\frac{4}{2} \Rightarrow 2$

PQ: (4,1), (4,0), (3,2), (2,1)

(5) (4,0) 3

howMany → | 2 | 1 | 1 |
0 1 2

$\frac{12}{4} \Rightarrow 3$

PQ: (4,0), (3,2), (3,0), (2,1)

(6) (3,2)

howMany → | 3 | 1 | -1 |
0 1 2

(3,2)

Ans.

PQ: (3,2), (3,0), (2,1)

Better Code:-

```
public static double minimiseMaxDistance (int arr[], int k){
    int n = arr.length;
    int howMany[] = new int[n-1];
    PriorityQueue<Pair> pq = new PriorityQueue<>((a,b) ->
                Double.compare(b.first, a.first));

    for (int i = 0; i<n-1; i++){                    → O(NlogN)
        pq.add(new Pair(arr[i+1] - arr[i], i));
    }

    for(int gasStations = 1; gasStations <= k; gasStations++){   → K
        Pair tp = pq.poll();
        int secInd = tp.second;

        howMany[secInd]++;

        double indiff = arr[secInd+1] - arr[secInd];
        double newSecLen = indiff / (double)(howMany[secInd]+1);
        pq.add(new Pair(newSecLen, secInd));          → logN
    }

    return pq.peek().first;
}
```

TC - O(NlogN + klogN)
SC - O(n-1) + O(n-1)

Optimal Dry-Run

PAGE NO.
DATE: / /

arr[] = [1, 2, 3, 4, 5], K = 4

0 0.1 0.2 0.3 0.4 0.5 0.6 ...

↳ we can not put it at distance 0 we'll need a lot of Statime to place them

⇒ 0.1 ⇒ [1, 1.1 1.2 1.3 1.4 1.5 1.6 1.7 ... 2

more than K

⇒ 0.2 ⇒ [1 1.2, 1.4, 1.6, 1.8, 2, 2.2, 2.4 ... 3]

more than K

⇒ 0.3 ⇒ [1 1.3 1.6 1.9 2 2.3 2.6, 2.9 ... 3]

more than K

⇒ 0.4 ⇒ [1 1.4 1.8 2 2.4 2.8 3 3.4 3.8 4, 4.4 ... 5]

more than K

⇒ 0.5 ⇒ [1 1.5 2 2.5 3 3.5 4 4.5 5]

= exactly K

Concept of opposite polarity will not work ..

```
public static int noOfStationsReq(double dist, int arr[]){
    int n = arr.length;
    int cnt = 0;
    for(int i=1; i<n; i++){
        int noInBetween = (int)((arr[i] - arr[i-1]) / dist);

        if((arr[i] - arr[i-1] == (dist * noInBetween)){
            noInBetween--;
        }

        cnt += noInBetween;
    }

    return cnt;
}

public static double minMaxDist(int arr[], int k){
    int n = arr.length; double low=0; double high=0;

    for(int i=0; i<n-1; i++){
        high = Math.max(high, (double)(arr[i+1] - arr[i]));
    }

    double diff = 1e-6;
    while(high - low > diff){
        double mid = (low + high) / (2.0);
        int cnt = noOfStationsReq(mid, arr);

        if(cnt > k){
            low = mid;
        }else{
            high = mid;
        }
    }
    return high;
}
```

TC - O(N * log(len)) + O(N)

SC - O(1)

25. Median of 2 Sorted arrays -

arr1[] = [1, 3, 4, 7, 10, 12] arr2[] = [2, 3, 6, 15]

Brute force Dry Run :-

arr3[] = | 1 | 2 | 3 | 3 | 4 | 6 | 7 | 10 | 12 | 15 |
0 1 2 3 4 5 6 7 8 9

$\frac{0+9}{2} \Rightarrow \frac{9}{2} \Rightarrow 4$ ✓ Done.

arr1[] = [1, 3, 4, 7, 10, 12] arr2[] = [2, 3, 6, 15]

i i j j j

i < j j++

① arr3 = | 1 | | .. |

i > j j++

② arr3 = | 1 | 2 | ... |

i > j j++

③ arr3 = | 1 | 2 | 3 | ... |

i < j i++

④ arr3 = | 1 | 2 | 3 | 3 | ... |

arr1 = [1, 3, 4, 7, 10, 12] arr2 = [2, 3, 6, 5]

(5) i<j i++

arr3 = | 1 | 2 | 3 | 3 | 4 | .. |

(6) i>j j++

arr3 = | 1 | 2 | 3 | 3 | 4 | 6 | ... |

(7) i<j i++

arr3 = | 1 | 2 | 3 | 3 | 4 | 6 | 7 | ... |

(8) i<j i++

arr3 = | 1 | 2 | 3 | 3 | 4 | 6 | 7 | 10 | ... |

(9) i<j i++

arr3 = | 1 | 2 | 3 | 3 | 4 | 6 | 7 | 10 | 12 | ... |

(10) i out of bound

(j<M)

mid

arr3 = | 1 | 2 | 3 | 3 | 4 | 6 | 7 | 10 | 12 | 15 |

Brute force Code :-

```
public static double median(int a[], int b[]){
    // size of two given arrays
    int n1 = a.length;
    int n2 = b.length;

    List<Integer> arr3 = new ArrayList<>();
    // Apply the merge Step
    int i = 0, j = 0, k = 0;
    while ( i<n1 && j<n2 ) {
        if ( a[i] < b[j] ) {
            arr3.add ( a[i++] );
        }
        else {
            arr3.add ( b[j++] );
        }
    }
    // copy the left out elements
    while (i<n1) {
        arr3.add ( a[i++] );
    }
    while (j<n2) {
        arr3.add ( b[j++] );
    }
    // find the median
    int n = n1 + n2;
    if ( n%2 == 1 ) {
        return (double) arr3.get(n/2);
    }
```

```
double median = ((double) arr3.get(n/2) +
                 (double) arr3.get((n/2)-1)) / 2.0;

return median;
}
```

TC - O(n1+n2)
SC - O(n1+n2)

→ Better Day Run: —

arr1 [] = [1, 3, 4, 7, 10, 12]

arr2 [] = [2, 3, 6, 15]

⇒ arr1 [] = [1, 3, 4, 7, 10, 12]
(i) i

cnt
1

arr2 [] = [2, 3, 6, 15]
j

cnt = 1

PAGE NO.
DATE : / /

(2) arr1[] = [1, 3, 4, 7, 10, 12] arr2[] = [2, 3, 6, 15]
i j

i < j

cnt1 = 2

(3) arr1[] = [1, 3, 4, 7, 10, 12] arr2[] = [2, 3, 6, 15]
i j

i > j

cnt2 = 3

(4) arr1[] = [1, 3, 4, 7, 10, 12] arr2[] = [2, 3, 6, 15]
i j

i < j

cnt3 = 3

(5) arr1[] = [1, 3, 4, 7, 10, 12] arr2 = [2, 3, 6, 15]
i j

i < j

cnt4 = 4 mid-1

(6) arr1[] = [1, 3, 4, 7, 10, 12] arr2 = [2, 3, 6, 15]
i j

i > j

cnt5 = 6 mid

Better Approach Code:-

```
public static double median(int a[], int b[]) {
    // size of two given arrays
    int n1 = a.length;
    int n2 = b.length;

    int n = n1 + n2;  // total size
    // required indices
    int ind2 = n/2;
    int ind1 = ind2 - 1;
    int cnt = 0;
    int ind1el = -1;
    int ind2el = -1;

    // apply the merge step
    int i=0, j= 0;
    while (i < n1 && j < n2) {
        if (a[i] < b[j]) {
            if (cnt == ind1) {
                ind1el = a[i];
            }
            if (cnt == ind2) {
                ind2el = a[i];
            }
            cnt++;
            i++;
        }
```

```
    else {
        if (cnt == ind1) {
            ind1el = b[j];
        }
        if (cnt == ind2) {
            ind2el = b[j];
        }
        cnt++;
        j++;
    }
}
// copy the left-out elements
while (i < n1) {
    if (cnt == ind1) {
        ind1el = a[i];
    }
    if (cnt == ind2) {
        ind2el = a[i];
    }
    cnt++;
    i++;
}
while (j < n2) {
    if (cnt == ind1) {
        ind1el = b[j];
    }
    if (cnt == ind2) {
        ind2el = b[j];
    }
    cnt++;
    j++;
}
```

```
// find the median
if (n%2 == 1) {
    return (double) ind2el;
}

return (double) ((double) (ind1 + ind2el))/2.0;
}
```

TC - $O(n_1 + n_2)$
SC - $O(1)$

(Optimal) Code:-

```
public static double median(int a[], int b[]){
    int n1 = a.length;
    int n2 = b.length;

    // if n1 is bigger swap the arrays
    if (n1 > n2) return median(b, a);

    int n = n1 + n2;
    int left = (n1 + n2 + 1) / 2;

    int low = 0;
    int high = n1;

    while (low <= high) {
        int mid1 = (low + high) / 2;
        int mid2 = left - mid1;

        // calculate l1, l2, r1 and r2
        int l1 = (mid1 > 0) ? a[mid1 - 1] :
                 Integer.MIN_VALUE;
        int l2 = (mid2 > 0) ? b[mid2 - 1] :
                 Integer.MIN_VALUE;

        int r1 = (mid1 < n1) ? a[mid1] :
                 Integer.MAX_VALUE;
        int r2 = (mid2 < n2) ? b[mid2] :
                 Integer.MAX_VALUE;
```

```
if ( l1 <= r2 && l2 <= r1 ) {
    if ( n%2 == 1 ) {
        return Math.max (l1, l2);
    }
    else {
        return (double) (Math.max (l1, l2)
                + Math.min (r1, r2))) / 2.0;
    }
}
else if ( l1 > r2 ) {
    high = mid1 - 1;
}
else {
    low = mid1 + 1;
}
}
return 0; // dummy statement
}
```

TC - O(log(min(n1, n2)))

SC - O(1)

26. K^{th} Element of 2 Sorted arrays -

~~nums1 = a_1, a_2, a_3~~

~~nums2 = b_1, b_2, b_3, b_4~~

arr1[] = [2, 3, 6, 7, 9]

arr2[] = [1, 4, 8, 10] k = 4

Brute force Dry Run:-

	0	1	2	3	4	5	6	7	8
arr3[] =	1	2	3	4	6	7	8	9	10

Return arr[k-1];

① if arr1[i] < arr2[j]

~~arr3[]~~ arr3.add(arr1.get(i));

② if arr1[i] > arr2[j]

arr3.add(arr2.get(j));

Brute force Code:-

```
public static int kthElement (ArrayList<Integer> a,
        ArrayList<Integer> b, int m, int n, int k){

    ArrayList<Integer> arr3 = new ArrayList<>();

    //apply the merge step:
    int i = 0, j = 0;

    while( i<m && j<n){
        if (a.get(i) < b.get(j)) {
            arr3.add (a.get(i++));
        }
        else {
            arr3.add (b.get(j++));
        }
    }

    //copy the left-out elements
    while (i<m) {
        arr3.add (a.get(i++));
    }
    while (j<n) {
        arr3.add (b.get(j++));
    }

    return arr3.get(k-1);
}
```

TC- O(m+n)
SC- O(m+n)

PAGE NO.
DATE: / /

-Better Approach Code:-

```
public static int kthElement (ArrayList<Integer> a,
        ArrayList<Integer> b, int m, int n, int k) {

    int ele = -1;
    int cnt = 0;        // counter
    // apply the merge step
    int i = 0, j = 0;

    while ( i<m && j<n ) {
        if (a.get(i) <= b.get(j)) {
            if (cnt == k-1) {
                ele = a.get(i);
            }
            cnt++;
            i++;
        }
        else {
            if (cnt == k-1) {
                ele = b.get(j);
            }
            cnt++;
            j++;
        }
    }
```

```
// copy the left-out elements
while (i <= m) {
    if (cnt == k-1) {
        ele = a.get(i);
    }
    cnt++;
    i++;
}

while (j <= n) {
    if (cnt == k-1) {
        ele = b.get(j);
    }
    cnt++;
    j++;
}

return ele;
}
```

TC- O(m+n)
SC- O(1)

PAGE NO.
DATE: / /

Optimal Code:-

```
public static int kthElement (ArrayList<Integer> a,
        ArrayList<Integer> b, int m; int n, int k){

    if (m>n){
        return kthElement (b,a, n, m, k);

        int left = k; // length of left half

        //apply binary search
        int low = Math.max (0, k-n);
        int high = Math.min (k, m);

        ~~while~~
    while (low <= high){
        int mid1 = (low+high) >>1;
        int mid2 = left - mid1;

        //calculate l1, l2, r1, and r2
        int l1 = Integer.MIN_VALUE;
        int l2 = Integer.MIN_VALUE;
        int r1 = Integer.MAX_VALUE;
        int r2 = Integer.MAX_VALUE;

        if (mid1 < m){
            r1 = a.get(mid1);
        }
        if (mid2 < n){
            r2 = b.get(mid2);
        }
```

```
if (mid1 - 1 >= 0) {
    l1 = a.get(mid1 - 1);
}
if (mid2 - 1 >= 0) {
    l2 = b.get(mid2 - 1);
}

if (l1 <= r2 && l2 <= r1) {
    return Math.max(l1, l2);
}

// eliminate the halves

else if (l1 > r2) {
    high = mid1 - 1;
}
else {
    low = mid1 + 1;
}
}
return 0;    // dummy statement
}
```

TC - O(log(min(m, n)))

SC - O(1)

PAGE NO
DATE / /

27. Find the row with maximum number of 1's :-

mat =

0	0	1	1	1	→ 3 1's
0	0	0	0	0	→ 0 1's
0	1	1	1	1	→ 4 1's
0	0	0	0	0	→ 0 1's
0	1	1	1	1	→ 4 1's

ans = 2 (2nd row).

Brute:- i=0 j=0 cnt_max = 0 idx = -1

(1) cnt_ones = mat[0][0] = 0
j=1 cnt_ones = mat[0][1] = 0
j=2 cnt_ones = mat[0][2] = 1
j=3 cnt_ones = mat[0][3] = 2
j=4 cnt_ones = mat[0][4] = 3

cnt_ones (3) > cnt_max (0)
cnt_max = 3
idx = 0

(2) i=1 j=0 cnt_max = 3 idx = 0 cnt_ones = 0
j=1 cnt_ones = mat[1][1] = cnt_ones = 0
j=2 cnt_ones = mat[1][2] = 0
j=3 cnt_ones = mat[1][3] = 0
j=4 cnt_ones = mat[1][4] = 0
cnt_ones < max

(3) i=2 j=0 cnt_ones = 0
j=1 cnt_ones = 1
j=2 cnt_ones = 2
j=3 cnt_ones = 3
j=4 cnt_ones = 4

Brute force Code:-

```
public static int rowWithMax1s(ArrayList<ArrayList
        <Integer>> matrix, int n, int m) {
    int cnt_max = 0;
    int index = -1;

    // traverse the matrix
    for(int i=0; i<n; i++){
        int cnt_ones = 0;
        for (int j=0; j<m; j++){
            cnt_ones += matrix.get(i).get(j);
        }
        if (cnt_ones > cnt_max) {
            cnt_max = cnt_ones;
            index = i;
        }
    }
    return index;
}
```

TC- O(n x m)
SC- O(1)

Optimal Code Dry Run :- ~~cntones = 0~~ cntmax = 0

① lowerBound (arr[0], m, 1) ⇒ 2

m-2 ⇒ 5-2 ⇒ 3

cntones (2) > cntmax

cntmax = 2

idx = 0

② lowerBound (arr[1], m, 1) ⇒ Hypothetical

(i) low = 0 high = n-1

mid = 0 + 5/2 ⇒ 2

arr[mid] = = 0 < x

low = mid+1

Each Row is Sorted

First Occurence

(ii) low = 3 high = 5

mid = 8/2 ⇒ 4

arr[mid] = = 0 < x

low = mid+1

(iii) low = 5 high = 5

mid = 10/2 ⇒ 5

arr[mid] = = 0 < x

low = mid+1 (exceeded)

return ans = = 5

m-5 = ⇒ 5-5 ⇒ 0

Cntmax (3) < ans

③ i = 2

Optimal Code:-

```
public static int lowerBound(ArrayList<Integer>
        arr, int n, int x) {
    int low = 0;
    int high = n-1;
    int ans = n;

    while (low <= high) {
        int mid = (low + high) / 2;
        //maybe an answer
        if (arr.get(mid) >= x) {
            ans = mid;
            //look for smaller idx on the left
            high = mid-1;
        }
        else {
            low = mid+1; //look on the right
        }
    }
    return ans;
}

public static int rowWithMax1s(ArrayList<ArrayList
        <Integer>> matrix, int n, int m) {

    int cnt_max = 0;
    int index = -1;
```

```
//traverse the rows:
for (int i = 0; i < n; i++) {
    //get the number of 1's -
    int cnt_ones = m - lowerBound(matrix.get(i),
                                  m, 1);
    if (cnt_ones > cnt_max) {
        cnt_max = cnt_ones;
        index = i;
    }
}
return index;
}
```

TC- O(n x logm)

SC- O(1)

28. Search in a 2D matrix -

	0	1	2	3
0	3	4	7	9
1	12	13	16	18
2	20	21	23	29

matrix =

target = 23

Brute force Dry-Run :-

(1) i = 0

j = 0	matrix[0][0]	!= 23 target
j = 1	matrix[0][1]	!= 23
j = 2	matrix[0][2]	!= 23
j = 3	matrix[0][3]	!= 23

(2) i = 1

j = 0	matrix[1][0]	!= 23 target
j = 1	matrix[1][1]	!= 23
j = 2	matrix[1][2]	!= 23
j = 3	matrix[1][3]	!= 23

(3) i = 2

j = 0	matrix[2][0]	!= target
j = 1	matrix[2][1]	!=
j = 2	matrix[2][2]	== target (23)
j = 3	matrix[2][3]	!=

return true :

Ans :

PAGE NO.
DATE: / /

Brute force Code:-

```
public static boolean searchMatrix (ArrayList<ArrayList
    <Integer>> matrix, int target) {
    int n = matrix.size();
    int m = matrix.get(0).size();

    //traverse the matrix
    for (int i= 0; i<n; i++ ) {
        for (int j= 0; j<m; j++) {
            if (matrix.get(i).get(j) == target) {
                return true;
            }
        }
    }
    return false;
}
```

TC- O(N×M)
SC- O(1)

Better Solution:-

(23) → target

3	4	7	9
12	13	16	18
20	21	23	29

(1) i = 0

matrix[0][m-1] > target &&
matrix[i][m-1] > target

i++

(2) i = 1

matrix[1][0] > target &&
matrix[i][m-1] > target

i++

(3) i = 2

matrix[2][0] > target &&
matrix[i][m-1] < target

{ lies btw this range 20 → 29 }

BinarySearch(arr[i], target)

low = 0 high = 4

mid = 4/2 ⇒ 2

arr[2] == target

ans = mid return true.

return ans

3 → 9 ✗

12 → 18 ✗

20 → 29 ✓

Find using BS

Better Approach Code:-

```
public static boolean binarySearch(ArrayList<Integer> nums,
        int target) {
    int n = nums.size();
    int low = 0, high = n-1;

    //perform the steps
    while(low <= high){
        int mid = low + (high-low)/2;

        if(nums.get(mid) == target) {
            return true;
        }
        else if(target > nums.get(mid)) {
            low = mid+1;
        }
        else {
            high = mid-1;
        }
    }
    return false;
}

public static boolean searchMatrix(ArrayList<
        ArrayList<Integer>> matrix, int target){
    int n = matrix.size();
    int m = matrix.get(0).size();
```

```
for (int i = 0; i<n; i++) {
    if (matrix.get(i).get(0) <= target &&
        target <= matrix.get(i).get(m-1)) {
        return binarySearch(matrix.get(i), target);
    }
}

return false;
}
```

TC- $O(N + \log M)$ ≈ $O(N)$

SC- $O(1)$

Optimal Code Dry Run:-

① Hypothetically flatten a 2D into 1D array

$$\text{matrix} = \begin{bmatrix} 3 & 4 & 7 & 9 \\ 12 & 13 & 16 & 18 \\ 20 & 21 & 23 & 29 \end{bmatrix} \quad \text{target} = 23$$

⇓

[3, 4, 7, 9, 12, 13, 16, 18, 20, 21, 23, 29] {Hypothetically}
0 1 2 3 4 5 6 7 8 9 10 11

TC - $O(\log_2(\text{size of arr } N * m))$

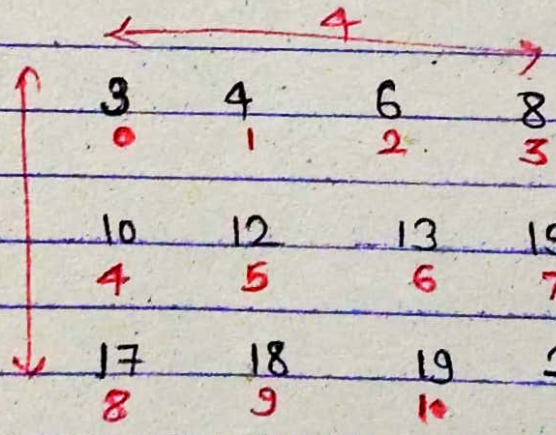
Convert the 1D → 2D (1,1)

5 (idx) = idx/m = row = 5/4 = 1
idx % m = col = 5%4 = 1

0 (low) 5 (mid) 11 (high)

m = No. of Columns

Intution behind formula -



row = has m no. of elements {first row is multiple of 4} ⇒ /m

col = m no. of elements has gone {remainder} %m

Optimal Code:-

```
public static boolean searchMatrix(ArrayList<
        ArrayList<Integer>> matrix, int target) {
    int n = matrix.size();
    int m = matrix.get(0).size();

    // apply binary search -
    int low = 0;
    int high = n*m - 1;

    while (low <= high) {
        int mid = low + (high - low)/2;
        int row = mid/m;
        int col = mid % m;

        if (matrix.get(row).get(col) == target) {
            return true;
        }
        else if (matrix.get(row).get(col) < target) {
            low = mid+1;
        }
        else {
            high = mid-1;
        }
    }
    return false;
}
```

TC - O(log(N×M))
SC - O(1)

PAGE NO.
DATE: / /

Search in a 2D Matrix - II

29. Search in a row and column wise sorted matrix -

	0	1	2	3	4
0	1	4	7	11	15
1	2	5	8	12	19
2	3	6	9	16	22
3	10	13	(14)	17	24
4	18	21	23	26	30

target = 14

ans ⇒ {3, 2}

Brute-force Dry Run:-

(1) i=0 j=0

if mat[0][0] == (1) != 14

j=1 mat[0][1] == (4) != 14

j=2 mat[0][2] == (7) != 14

j=3 mat[0][3] == (11) != 14

j=4 mat[0][4] == (15) != 14

(2) i=1 j=0 (2) != 14

j=1 (5) != 14

j=2 (8) != 14

j=3 (12) != 14

j=4 (19) != 14

(3) i=2 j=0 (3) != (3)

j=1 (6) != (6)

j=2 (9) != (9)

j=3 (16) != (16)

j=4 (22) != (22)

Brute force code:-

```
public static boolean searchElement (ArrayList<ArrayList
        <Integer>> matrix, int target) {
    int n = matrix.size();
    int m = matrix.get(0).size();

    // traverse the matrix
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            if (matrix.get(i).get(j) == target) {
                return true;
            }
        }
    }
    return false;
}
```

TC- O(N x M)
SC- O(1)

Better Dry Run :- target = 5

(1) i = 0 {Why are we not using conditional check for range? because I have sorted column and it is not necessary that the last element of each row is less than the first element of next row?}
binarySearch(mat[0], target)

(2) i = 1
binarySearch(mat[1], target)

return true;

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24

flag == true.
return true As an Answer...

- Integers in each row are sorted in ascending from left to right
- Integers in each column are sorted in ascending from top to bottom.

In 2D-matrix I :-
- Each row is sorted in non-decreasing order
- The first integer of each row is sorted than the last integer of the previous row.

Better Code:-

```
public static boolean binarySearch(ArrayList<Integer>
    nums, int target) {
    int n = nums.size();
    int low = 0, high = n-1;

    // perform the steps
    while(low <= high) {
        int mid = (low + high)/2;

        if(nums.get(mid) == target) {
            return true;
        }
        else if( target > nums.get(mid)) {
            low = mid+1;
        }
        else {
            high = mid-1;
        }
    }
    return false;
}

public static boolean searchElement(ArrayList<
    ArrayList<Integer>> matrix, int target){
    int n = matrix.size();
    int m = matrix.get(0).size();
```

```
for (int i=0; i<n; i++) {
    boolean flag = binarySearch(matrix.get(i), target);
    if (flag == true) {
        return true;
    }
}
return false;
}
```

TC- O(N*logM)
SC- O(1)

Optimal Dry Run:-

	0	1	2	3	4
0	1	4	7	11	15
1	2	5	8	12	19
2	3	6	9	16	22
3	10	13 found	14	17	24
4	18	21	23	26	30

target = 13

① left = 0 right = m-1

mat[0][4] == 15 > 13

right --

② mat[0][3] == 11 < 13

left ++

③ mat[1][3] == 12 < 13

left ++

④ mat[2][3] == 16 > 13

right --

⑤ mat[2][2] == 9 < 13

left ++;

⑥ mat[3][2] == 14 > 13

right --

⑦ mat[3][1] == 13

return true

If I stand in 1
I cannot decide wether to go because left → and down ↓ both are increasing

If I stand in 30
I cannot decide wether to go because ↑ top ← left for 30 both are decreasing.

(18)

If I stand in 15
I can decide wether to go because ← left is decreasing and ↓ bottom is increasing

PAGE NO.
DATE:

Optimal Code :-

```
public static boolean searchElement (ArrayList<ArrayList
        <Integer>> matrix, int target){
    int n = matrix.size();
    int m = matrix.get(0).size();

    int row = 0;
    int col = m-1;

    // traverse the matrix from (0, m-1);
    while(row<n && col >= 0){
        if(matrix.get(row).get(col) == target){
            return true;
        }
        else if(matrix.get(row).get(col) < target){
            row++;
        }
        else{
            col--;
        }
    }
    return false;
}
```

TC - O(N+M)
SC - O(1)

30. Find Peak Element in a 2D matrix -

	0	1	2	3	4	5
0	4	2	5	1	4	5
1	2	9	3	2	3	2
2	1	7	6	0	1	3
3	3	6	2	3	7	2

ans = 4

Brute ⇒ Go in Every Element and check for 4 directions if they are smaller or not.

TC - O((N * M) * 4)

⇓ largest of matrix

Better ⇒ finding the largest of matrix can be my peak element.

TC - O (N * M)

~~Better~~ Optimal :-

0 —— 2 —— 5

left mid right

in 2nd col find largest which is -

7 > 6 > 0 { there is someone who is greater than 6 }

Eliminate right half

2

0 —— 1 — mid

left high

0 —— 1 high

left

mid

max = 4

Code /

```
private int findMaxIndex(int[][] mat, int n, int m,
                         int col) {
    int maxValue = -1;
    int index = -1;

    for(int i = 0; i<n; i++) {
        if(mat[i][col] >= maxValue) {
            maxValue = mat[i][col];
            index = i;
        }
    }
    return index;
}
```

TC - O(log(m*n))
SC - O(1)

```
public int[] findPeakGrid(int[][] mat) {
    int n = mat.length;
    int m = mat[0].length;
    int low = 0, high = m-1;
    while(low <= high) {
        int mid = (low + high)/2;
        int maxRowIdx = findMaxIndex(mat, n, m, mid);
        int left = (mid-1 >= 0)? mat[maxRowIdx][mid-1]:-1;
        int right = (mid+1 <m) ? mat[maxRowIdx][mid+1]:-1;

        if(mat[maxRowIdx][mid] >= left &&
           mat[maxRowIdx][mid] >= right) {
            return new int[] {maxRowIdx, mid};
        }else if(mat[maxRowIdx][mid] < left) {
            high = mid-1;
        }else {
            low = mid+1;
        }
    }
    return new int[] {-1,-1};
```

31. Matrix Median - {Row wise sorted}

1	5	7	9	11
2	3	4	5	10
9	10	12	14	16

n x m
↓ odd ↘ odd

Ans ⇒ 9

Brute - force Dry Run :-

① Go through each & every element in the matrix and store them into an Arraylist

list

1	5	7	9	11	2	3	4	5	10	9	10	12	14	16

⇓

Now Sort the Collection.

1	2	3	4	5	5	7	9	9	10	10	11	12	14	16

return $\frac{0 + high}{2}$ → (9) ✓

Brute:
~~Better~~ Code:-

```
public static int median(int mat[][], int m, int n){
    List<Integer> lst = new ArrayList<>();

    // Traverse the matrix and
    // copy the elements to the list
    for(int i=0; i<m; i++){
        for(int j=0; j<n; j++){
            lst.add(mat[i][j]);
        }
    }

    // sort the list:
    Collections.sort(lst);
    return lst.get((m*n)/2);
}
```

TC- O(MXN(log(MXN))) + O(MXN)

SC- O(MXN)

Optimal Dry Run:-

1	5	7	9	11
2	3	4	5	10
9	10	12	14	16

(1) ⇒ lowest (16) ⇒ highest

#01 My answer will be between 1 — 16 for sure.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

<= 1 2 3 4 6 6 7 ~~7~~ 9 11 12 13 13 14 14 15

↳ lesser than || equal to

Optimal Code:-

```
Static int upperBound (int arr[], int x, int n) {
    int low = 0, high = n-1;
    int ans = n;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        // maybe an answer
        if (arr[mid] > x) {
            ans = mid;
            // look for a smaller idx on the left
            high = mid-1;
        }
        else {
            low = mid+1;
        }
    }
    return ans;
}

Static int countSmallEqual (int mat[][], int m,
                            int n, int x) {
    int cnt = 0;
    for (int i = 0; i<m; i++) {
        cnt += upperBound (mat[i], x, n);
    }
    return cnt;
}
```

```
static int median (int mat[][], int m, int n) {
    int low = Integer.MAX_VALUE;
    int high = Integer.MIN_VALUE;

    // point low and high to right elements
    for (int i = 0; i < m; i++) {
        low = Math.min (low, mat[i][0]);
        high = Math.max (high, mat[i][n-1]);
    }

    int req = (n * m) / 2;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int smallEqual = countSmallEqual (mat, m, n, mid);

        if (smallEqual <= req) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return low;
}
```

TC - O(log(10^9)) * O(M(log N))

SC - O(1)